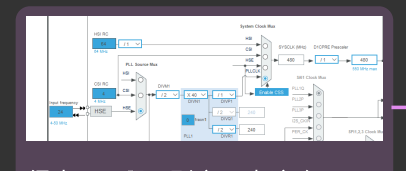
H7
工具链：cubemx
keil （V5编译器）
freemaster

时钟树配置注意事项



得走HSE啊，别忘了点这个

RTOS配置注意事项

systick-来源于cortex内核自带的systick定时器

RTOS常常使用systick定时器，hal最好就换一个，直接开一个别的定时器作为系统节拍来源

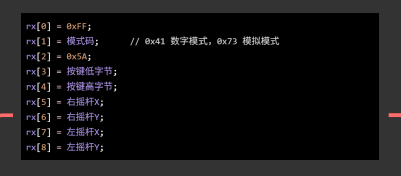
1.PS2遥控指令接收（L3 R3是摇杆按下）

接线：
电源：
VCC
GND
经典SPI：
CLK
CS
DI
DO
手柄应答：
ACK（先不接）

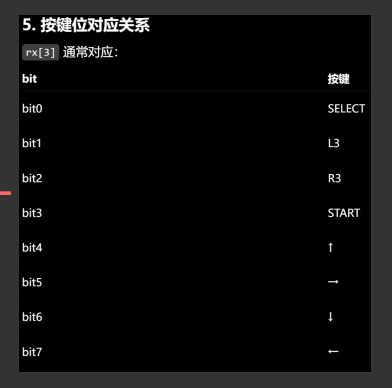
原理：
主机给手柄发送查询数据，手柄给主机就返回手柄状态数据。
符合SPI特点，可以简单认为通信过程双向同时，H7发一字节数据，手柄就给H7一字节数据
所以是H7发送请求手柄回复而不是手柄发送H7接受

mode分为数字模式和模拟模式
其实数字模式就是禁用摇杆的模拟模式

数据格式：



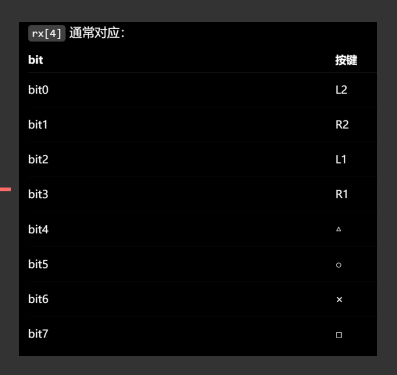
```
rx[0] = 0xFF;
rx[1] = 模式码;      // 0x41 数字模式, 0x73 模拟模式
rx[2] = 0x5A;
rx[3] = 按键低字节;
rx[4] = 按键高字节;
rx[5] = 右摇杆X;
rx[6] = 右摇杆Y;
rx[7] = 左摇杆X;
rx[8] = 左摇杆Y;
```



5. 按键位对应关系

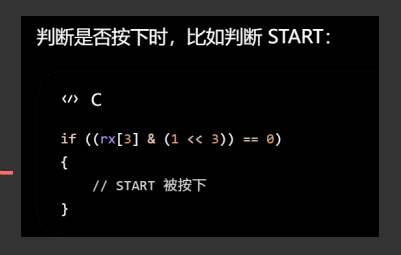
`rx[3]` 通常对应：

bit	按键
bit0	SELECT
bit1	L3
bit2	R3
bit3	START
bit4	↑
bit5	→
bit6	↓
bit7	←



`rx[4]` 通常对应：

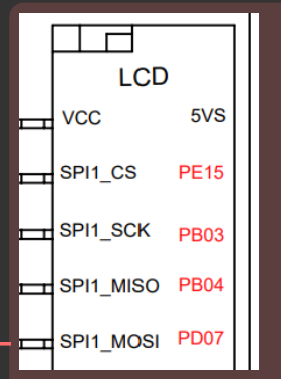
bit	按键
bit0	L2
bit1	R2
bit2	L1
bit3	R1
bit4	△
bit5	○
bit6	×
bit7	□



判断是否按下时，比如判断 START：

```c
if ((rx[3] & (1 << 3)) == 0)
{
    // START 被按下
}
```

读取的时候，记住每个字节从右向左，低位在右
遥感数据零位置的时候是128

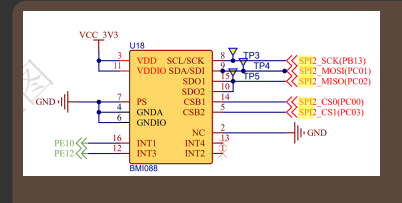


接线：使用LCD的口了，得找这种10Pin的接线端子
PD7 -- DO
PB4 -- DI
PB3 -- CLK
PE15 -- CS -- 低电平有效

5V
GND

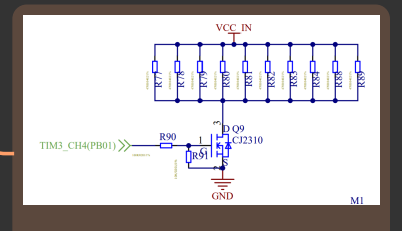
键码值

2.陀螺仪数据读取与温度控制



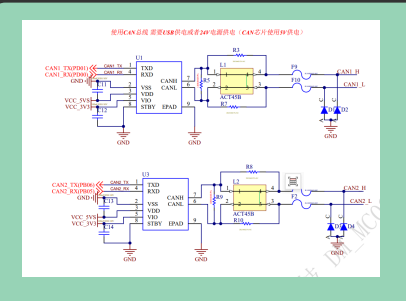
两个片选，因为加速度计和陀螺仪虽然在同一个封装下，但是是两套设备。
需要两套设备交替和H7通信
PE10 -- INT1
PE12 -- INT3

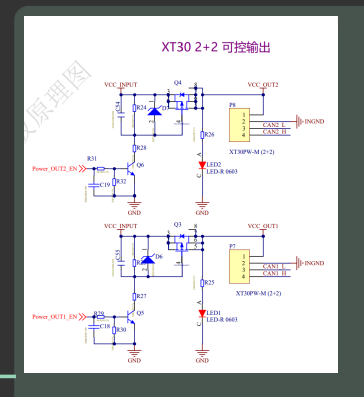
PB13 -- SPI2_SCK
PC1 -- SPI2_MOSI
PC2 -- SPI2_MISO
PC0 -- SPI2_CS0
PC3 -- SPI2_CS1



温度控制
PB1 -- TIM3_CH4

3.双路CAN电机指令发送与反馈读取



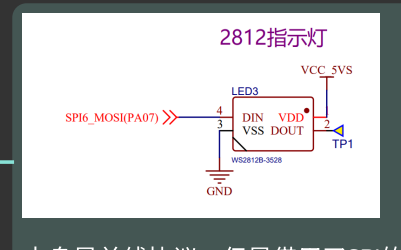


两个GPIO控制供电开关
Power_OUT2_EN -- PC13
Power_OUT1_EN -- PC14-OSC32_IN

PD0 -- CAN1_RX
PD1 -- CAN1_TX
PB5 -- CAN2_RX
PB6 -- CAN2_TX

其中，中断经典作用是提醒H7数据就绪
具体怎么实现的还得是看代码

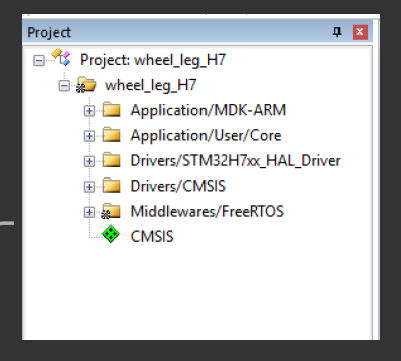
4.指示灯



本身是单线协议，但是借用了SPI的外设生成其时序

嵌入式软件

首先看文件树



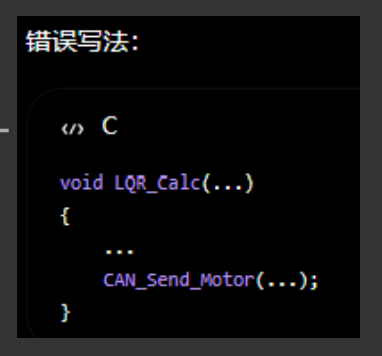
```
Project: wheel_leg_H7
  wheel_leg_H7
    Application/MDK-ARM
    Application/User/Core
    Drivers/STM32H7xx_HAL_Driver
    Drivers/CMSIS
    Middlewares/FreeRTOS
    CMSIS
```

主要学习下面达妙的业务代码分层
顺便理清代码模块结构

编写的代码存放规则：

1. main.c这个文件只做初始化，不要写业务逻辑

2. 硬件驱动和算法/控制算法分离，比如不要LQR函数里面直接调用发送CAN信号的命令

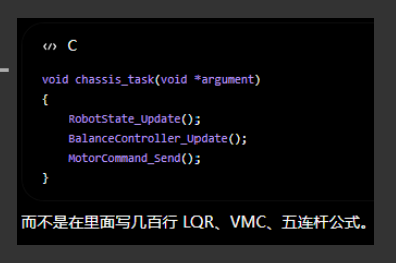


错误写法：

```c
void LQR_Calc(...)
{
    ...
    CAN_Send_Motor(...);
}
```

3. Algorithm不依赖HAL，不要依赖HAL库相关的东西，就看成存粹的C语言程序实现

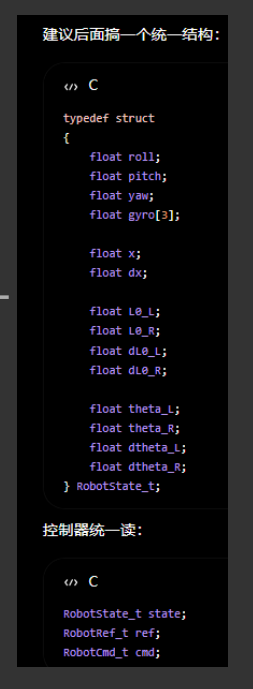
4. APP是调度着，属于领导，不能啥细节都管
正面例子：



```c
void chassis_task(void *argument)
{
    RobotState_Update();
    BalanceController_Update();
    MotorCommand_Send();
}
```

而不是在里面写几百行 LQR、VMC、五连杆公式。

5. 全局状态集中管理：



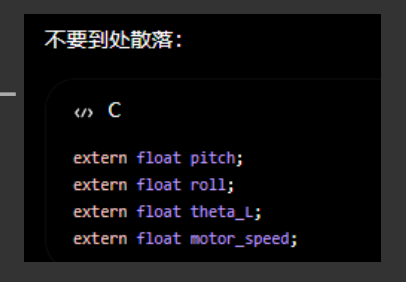
建议后面搞一个统一结构：

```c
typedef struct
{
    float roll;
    float pitch;
    float yaw;
    float gyro[3];

    float x;
    float dx;

    float L0_L;
    float L0_R;
    float dL0_L;
    float dL0_R;

    float theta_L;
    float theta_R;
    float dtheta_L;
    float dtheta_R;
} RobotState_t;
```

控制器统一读：

```c
RobotState_t state;
RobotRef_t ref;
RobotCmd_t cmd;
```

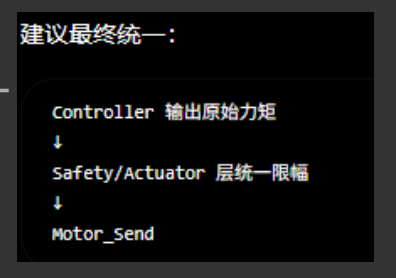
而不要：



不要到处散落：

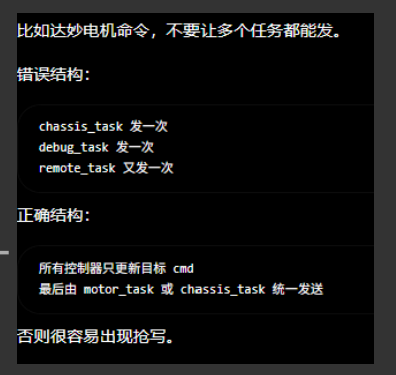
```c
extern float pitch;
extern float roll;
extern float theta_L;
extern float motor_speed;
```

6. 电机输出只在一个地方限幅
不然不好判断哪个环节超出安全限制了



建议最终统一：

```
Controller 输出原始力矩
↓
Safety/Actuator 层统一限幅
↓
Motor_Send
```



比如达妙电机命令，不要让多个任务都能发。

错误结构：

```
chassis_task 发一次
debug_task 发一次
remote_task 又发一次
```

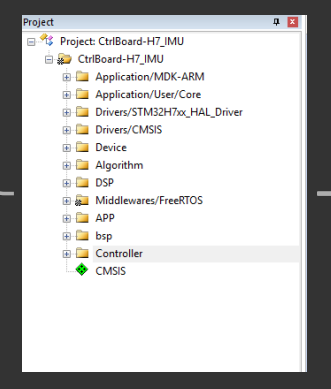
正确结构：

```
所有控制器只更新目标 cmd
最后由 motor_task 或 chassis_task 统一发送
```

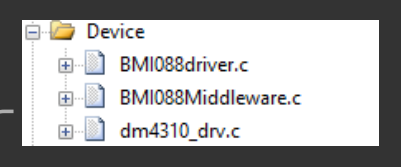
否则很容易出现抢写。

7. 不要让多个APP里面的任务都并联接入到最后执行器。正确结构是任务都把请求汇总到一个中间媒介，然后有一个任务专门接到执行器

8.（这个当下最有用）自顶向下，先搭建框架，后填补算法
这个以前都没这样做过，但是实践中确实应该这样做，这把高低试试

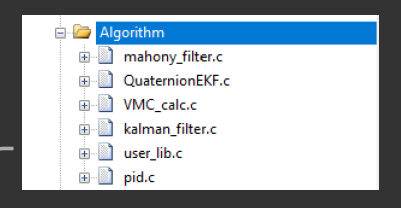


```
Project: CtrlBoard-H7_IMU
  CtrlBoard-H7_IMU
    Application/MDK-ARM
    Application/User/Core
    Drivers/STM32H7xx_HAL_Driver
    Drivers/CMSIS
    Device
    Algorithm
    DSP
    Middlewares/FreeRTOS
    APP
    bsp
    Controller
    CMSIS
```



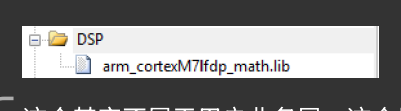
```
Device
  BMI088driver.c
  BMI088Middleware.c
  dm4310_drv.c
```

硬件设备层：把硬件设备封装成输入输出接口



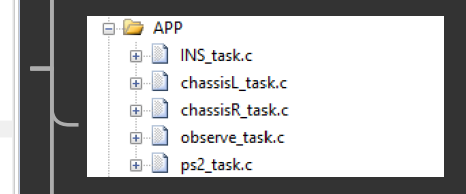
```
Algorithm
  mahony_filter.c
  QuaternionEKF.c
  VMC_calc.c
  kalman_filter.c
  user_lib.c
  pid.c
```

纯算法层：只做数学运算，不带工程风味，只是一味数学运算操作



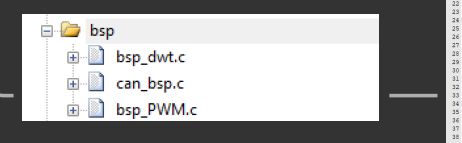
```
DSP
  arm_cortexM7lfdp_math.lib
```

这个其实不属于用户业务层，这个是生成的数学库相关分组。矩阵运算，滤波，FFT用到



```
APP
  INS_task.c
  chassisL_task.c
  chassisR_task.c
  observe_task.c
  ps2_task.c
```

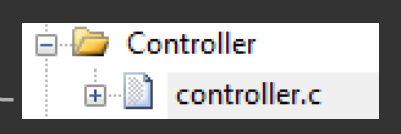
应用层：把设备和算法和控制器穿起来



```
bsp
  bsp_dwt.c
  can_bsp.c
  bsp_PWM.c
```

BSP：Board Support Package

把ST官方硬件外设接口进行一次针对任务的再封装



```
Controller
  controller.c
```

控制器层：控制视角下的控制器设计

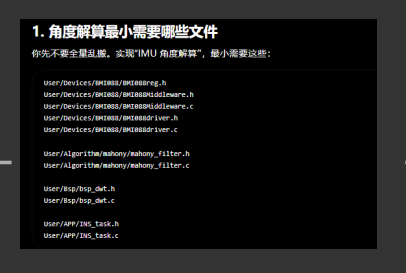
开始按照建议，先搭建框架，自顶向下编程

PS2

main.c 初始化 GPIO 和 SPI1
↓
FreeRTOS 创建 PS2_Task
↓
PS2_Task 里调用 PS2_Init()
↓
循环调用 PS2_Update()
↓
其他控制任务通过 PS2_GetData() 读取遥控器数据

设置的100HZ读取频率

IMU



1. 角度解算最小需要哪些文件

你先不要全量乱搬。实现"IMU 角度解算"，最小需要这些：

```
User/Devices/BMI088/BMI088reg.h
User/Devices/BMI088/BMI088Middleware.h
User/Devices/BMI088/BMI088Middleware.c
User/Devices/BMI088/BMI088driver.h
User/Devices/BMI088/BMI088driver.c

User/Algorithm/mahony/mahony_filter.h
User/Algorithm/mahony/mahony_filter.c

User/Bsp/bsp_dwt.h
User/Bsp/bsp_dwt.c

User/APP/INS_task.h
User/APP/INS_task.c
```

这个比较复杂，先按照文件树理解

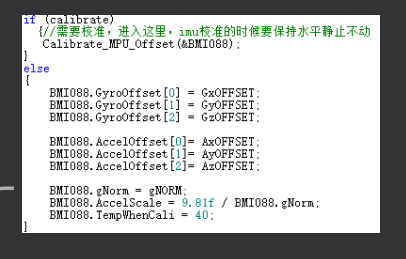
DWT：是M7内核自带的周期计数器
进行微秒级延时
可以用来测量代码运行时间
不需要在cubemx里面单独配置，这不是外设

按照初始化函数来理解

初始化两个设备

流程都大致是：
1.拉低CS（使能）
2.读取CHIP_ID，确认对对的设备操作
3.对设备软件复位
4.配置参数
5.回读配置，检查配置是否写入成功

可选择的校准



```c
if (calibrate)
  {//需要校准，进入这里，imu校准的时候要保持水平静止不动
   Calibrate_MPU_Offset(&BMI088);
}
else
{
    BMI088.GyroOffset[0] = GxOFFSET;
    BMI088.GyroOffset[1] = GyOFFSET;
    BMI088.GyroOffset[2] = GzOFFSET;

    BMI088.AccelOffset[0]= AxOFFSET;
    BMI088.AccelOffset[1]= AyOFFSET;
    BMI088.AccelOffset[2]= AzOFFSET;

    BMI088.gNorm = gNORM;
    BMI088.AccelScale = 9.81f / BMI088.gNorm;
    BMI088.TempWhenCali = 40;
}
```

得到校准参数的过程
上面是再次校准，下面是加载默认参数了
可以看见达妙也是在40度下校准的

1. 陀螺仪零偏
把静止时候的误差求平均
在真实数据上减去这个误差
目的是让静止时的角速度接近0

2.重力模长
目的是修正加速度计比例误差，
让静止时的重力模长回到9.81

3.加速度计零偏
同1

校准参数应该是有关温度的函数

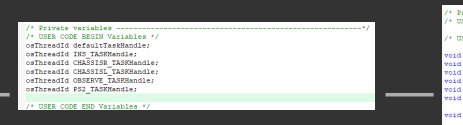
达妙给的默认校准参数写了是在40度情况下得到的。所以严格来说使用这套参数，需要用温控控制40度恒温

对源代码做出改动，采用AI写的.c里面开静态变量对数据操作，对外开放一个只读指针接口

INS任务里面是把设备驱动，也就是读取到原始数据的代码和解算代码组合在一起使用，最后得到欧拉角，这个是最后需要的数据

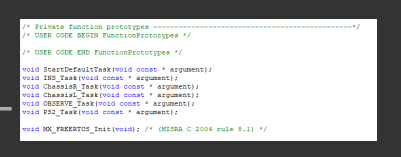
探索程序

freertos.c 基本就是 RTOS 任务总调度入口，负责初始化任务



```c
/* Private variables ---------------------------------------------------------*/
/* USER CODE BEGIN Variables */
osThreadId defaultTaskHandle;
osThreadId INS_TASKHandle;
osThreadId CHASSISR_TASKHandle;
osThreadId CHASSISL_TASKHandle;
osThreadId OBSERVE_TASKHandle;
osThreadId PS2_TASKHandle;

/* USER CODE END Variables */
```

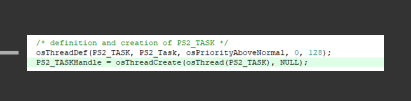
给任务分配ID-身份证号



```c
/* Private function prototypes -----------------------------------------------*/
/* USER CODE BEGIN FunctionPrototypes */

/* USER CODE END FunctionPrototypes */

void StartDefaultTask(void const * argument);
void INS_Task(void const * argument);
void ChassisR_Task(void const * argument);
void ChassisL_Task(void const * argument);
void OBSERVE_Task(void const * argument);
void PS2_Task(void const * argument);

void MX_FREERTOS_Init(void); /* (MISRA C 2004 rule 8.1) */
```

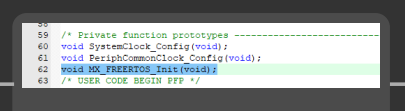
下面是定义任务函数，给任务分配活

下面写一个初始化所有任务的入口函数
MX_FREERTOS_Init，里面包含了所有任务的定义以及创建



```c
/* definition and creation of PS2_TASK */
osThreadDef(PS2_TASK, PS2_Task, osPriorityAboveNormal, 0, 128);
PS2_TASKHandle = osThreadCreate(osThread(PS2_TASK), NULL);
```

C语言的链接机制



```c
/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
void PeriphCommonClock_Config(void);
void MX_FREERTOS_Init(void);
/* USER CODE BEGIN PFP */
```

mian.c里面已经声明了

然后在freertos.c中直接写这个函数的实现，不用在freertos.h里面写实现，就能直接在main.c里面调用这个函数

代码框架测试实验

- 1.PS2通信
- 2.IMU数据读取
- 3.IMU温度控制
- 细分主题 4

Presented with xmind